

FROM TASK 2 - CREATE TABLE Statements

Hibernate:

```
create table transactions (  
    amount float(53) not null,  
    transaction_id bigint generated by default as identity,  
    note varchar(255),  
    receiver_upi_id varchar(255) not null,  
    sender_upi_id varchar(255) not null,  
    primary key (transaction_id)  
)
```

Hibernate:

```
create table users (  
    balance float(53) not null,  
    user_id bigint generated by default as identity,  
    email varchar(255) not null unique,  
    name varchar(255) not null,  
    phone_number varchar(255) unique,  
    upi_id varchar(255) not null unique,  
    primary key (user_id)  
)
```

FROM TASK 2 - H2 Console Screenshots (BEFORE data insertion)

```
SELECT * FROM USERS;
```

```
|SELECT * FROM USERS|
```

```
SELECT * FROM USERS;
```

BALANCE	USER_ID	EMAIL	NAME	PHONE_NUMBER	UPI_ID
---------	---------	-------	------	--------------	--------

(no rows, 2 ms)

```
SELECT * FROM TRANSACTIONS;
```

```
|SELECT * FROM TRANSACTIONS|
```

```
SELECT * FROM TRANSACTIONS;
```

AMOUNT	TRANSACTION_ID	NOTE	RECEIVER_UPI_ID	SENDER_UPI_ID
--------	----------------	------	-----------------	---------------

(no rows, 2 ms)

SECTION 5: FROM TASK 5 - cURL Commands & Output

Test 1: POST /users (Register User 1)

Response:

```
$ curl -X POST http://localhost:8080/users \
-H "Content-Type: application/json" \
-d '{
  "name": "Alice",
  "email": "alice@test.com",
  "upiId": "alice@okaxis",
  "phoneNumber": "9876543210",
  "balance": 10000
}'
{"userId":1,"name":"Alice","email":"alice@test.com","upiId":"alice@okaxis","phoneNumber":"9876543210","balance":10000.0}
```

Test 2: POST /users (Register User 2)

Response:

```
$ curl -X POST http://localhost:8080/users \
-H "Content-Type: application/json" \
-d '{
  "name": "Bob",
  "email": "bob@test.com",
  "upiId": "bob@okaxis",
  "phoneNumber": "9876543211",
  "balance": 5000
}'
{"userId":3,"name":"Bob","email":"bob@test.com","upiId":"bob@okaxis","phoneNumber":"9876543211","balance":5000.0}
```

Test 3: GET /users (Get All Users)

Response:

```
$ curl -X GET http://localhost:8080/users
[{"userId":1,"name":"Alice","email":"alice@test.com","upiId":"alice@okaxis","phoneNumber":"9876543210","balance":10000.0}, {"userId":2,"name":"Bob","email":"bob@test.com","upiId":"bob@okaxis","phoneNumber":"9876543211","balance":5000.0}]
```

Test 4: POST /transactions (Send Money)

Response:

```
$ curl -X POST http://localhost:8080/transactions \
-H "Content-Type: application/json" \ ...
{"senderUpiId": "alice@okaxis",
 "receiverUpiId": "bob@okaxis",
 "amount": 1000,
 "note": "Payment"
}'
{"transactionId":1,"senderUpiId":"alice@okaxis","receiverUpiId":"bob@okaxis","amount":1000.0,"note":"Payment"}
```

FROM TASK 5 - H2 Console Screenshots (AFTER data insertion)

[SCREENSHOT 3: H2 Console - Users Table (with data)]

SELECT * FROM USER;

SELECT * FROM USERS

SELECT * FROM USERS;

BALANCE	USER_ID	EMAIL	NAME	PHONE_NUMBER	UPI_ID
9000.0	1	alice@test.com	Alice	9876543210	alice@okaxis
6000.0	2	bob@test.com	Bob	9876543211	bob@okaxis

(2 rows, 1 ms)

Show Alice and Bob with updated balances

[SCREENSHOT 4: H2 Console - Transactions Table (with data)]

SELECT * FROM TRANSACTION;

SELECT * FROM TRANSACTIONS

SELECT * FROM TRANSACTIONS;

AMOUNT	TRANSACTION_ID	NOTE	RECEIVER_UPI_ID	SENDER_UPI_ID
1000.0	1	Payment	bob@okaxis	alice@okaxis

(1 row, 0 ms)

Show the transaction record

SECTION 7: FROM TASK 5 - @RequestBody Demonstration

WITHOUT @RequestBody

```
$ curl -X POST http://localhost:8080/users -H "Content-Type: application/json" -d '{"name":"Alice","email":"alice@test.com","phoneNumber":"9876543210","balance":10000}'  
{"timestamp":"2026-06-13T17:53:29.427Z","status":500,"error":"Internal Server Error","path":"/users"}
```

SECTION 8: FROM TASK 6 - Custom Queries

GET /users/upi/{upild} Endpoint Test

Response:

```
$ curl http://localhost:8080/users/upi/alice@okaxis  
{ "userId":1, "name": "Alice", "email": "alice@test.com", "upiId": "alice@okaxis", "phoneNumber": "9876543210", "balance": 9000.0 }
```

SECTION 9: Conceptual Write-Up Answers (6 questions)

Question 1: Request Lifecycle

Trace what happens from the moment curl sends POST /users to the moment your registerUser method runs. Name the DispatcherServlet and Handler Adapter in your answer.

Request Lifecycle - DispatcherServlet intercepts the request, HandlerAdapter deserializes JSON via Jackson, routes to your controller method

Question 2: Serialisation

When you POST a JSON payload like {"name": "Priya", "upild": "priya@okaxis"}, what converts it into a Java User object? What happens if the JSON key is "upi_id" instead of "upild"?

Serialisation - Jackson's ObjectMapper converts JSON to Java objects; wrong key names cause mapping failures or are ignored

Question 3: Spring Boot Features

Name the three Spring Boot features. For each one, point to a specific thing in your PayFlow project where that feature is doing work for you.

Spring Boot Features - Auto-configuration (H2 setup), Embedded Server (Tomcat), Starter Dependencies (spring-boot-starter-data-jpa)

Question 4: Spring vs. Spring Boot

If you had used plain Spring instead of Spring Boot for PayFlow, what would you have had to set up manually? What does Spring Boot take care of automatically?

Spring vs Spring Boot - Plain Spring needs manual XML config, bean definitions, application server setup; Boot handles all automatically

Question 5: Stateless REST

Your POST /transactions endpoint does not remember anything about the previous request. What does stateless mean, and why does it matter if PayFlow eventually runs on three servers behind a load balancer?

Stateless REST - Each request is independent (no session); critical for load balancers to route requests to any server without losing context

Question 6: Persistence

You stored transactions in the H2 database, not in a Java List. What would have happened to all the transaction records if you had used a List and then restarted the server? Why is this unacceptable for a payments app?

Persistence - Java List loses data on restart (unacceptable); H2 database ensures durability required for payments compliance